2.2.1

```
(define nth-elt
  (lambda (lst n)
    (if (null? lst)
        (error "nth-elt: list too short")
        (if (pair? lst)
            (if (zero? n)
                (car lst)
                (nth-elt (cdr lst) (- n 1)))
            (error "nth-elt: not a list")))))

(define list-length
  (lambda (lst)
    (if (null? lst)
        0
        (if (pair? lst)
            (+ 1 (list-length (cdr lst)))
            (error "list-length: not a list")))))
```

2.2.5

```
(define subst
  (lambda (new old slst)
    (map (lambda (sexp) (subst-sexp new old sexp)) slst)))

(define subst-sexp
  (lambda (new old sexp)
    (if (symbol? sexp)
        (if (eq? old sexp) new sexp)
        (subst new old sexp))))
```

2.2.7

1

```
(define duple
```

```
  (lambda (n x)
    (if (zero? n)
        '()
        (cons x (duple (- n 1) x)))))
```

2

```
(define invert
  (lambda (lst)
    (if (null? lst)
        '()
        (cons (list (cadar lst) (caar lst))
              (invert (cdr lst))))))
```

3

```
(define list-index
  (lambda (s los)
    (li s los 0)))

(define li
  (lambda (s los index)
    (if (null? los)
        -1
        (if (eq? s (car los))
            index
            (li s (cdr los) (+ index 1))))))
```

4

```
(define vector-index
  (lambda (s vos)
    (vi s vos (vector-length vos))))

(define vi
  (lambda (s vos index)
    (if (zero? index)
```

```
        -1
        (if (eq? s (vector-ref vos (- index 1)))
            (- index 1)
            (vi s vos (- index 1))))))
```

5

```
(define ribassoc
  (lambda (s los v fail-value)
    (ribassoc-iter s los v fail-value 0)))

(define ribassoc-iter
  (lambda (s los v fail-value count)
    (if (null? los)
        fail-value
        (if (eq? s (car los))
            (vector-ref v count)
            (ribassoc-iter
             s (cdr los) v fail-value (+ count 1))))))
```

6

```
(define filter-in
  (lambda (p lst)
    (if (null? lst)
        '()
        (if (p (car lst))
            (cons (car lst) (filter-in p (cdr lst)))
            (filter-in p (cdr lst))))))
```

7

```
(define product
  (lambda (los1 los2)
    (if (null? los1)
        '()
        (append (prdct (car los1) los2)
```

```
                (product (cdr los1) los2)))))

(define prdct
  (lambda (s los)
    (if (null? los)
        '()
        (cons (list s (car los)) (prdct s (cdr los))))))
```

8

```
(define swapper
  (lambda (s1 s2 slst)
    (if (null? slst)
        '()
        (cons (swap-in-sexp s1 s2 (car slst))
              (swapper s1 s2 (cdr slst))))))

(define swap-in-sexp
  (lambda (s1 s2 sexp)
    (if (symbol? sexp)
        (if (eq? s1 sexp)
            s2
            (if (eq? s2 sexp)
                s1
                sexp))
        (swapper s1 s2 sexp))))
```

9

```
(define rotate
  (lambda (los)
    (if (null? los)
        '()
        (cons (last los) (but-last los)))))
```

```
(define last
  (lambda (los)
    (if (null? los)
        '()
        (if (null? (cdr los))
            (car los)
            (last (cdr los))))))

(define but-last
  (lambda (los)
    (if (null? los)
        '()
        (if (null? (cdr los))
            '()
            (cons (car los) (but-last (cdr los)))))))
```

2.2.8

1

```
(define down
  (lambda (lst)
    (map list lst)))
```

2

```
(define up
  (lambda (lst)
    (if (null? lst)
        '()
        (if (symbol? (car lst))
            (cons (car lst) (up (cdr lst)))
            (append (car lst) (up (cdr lst)))))))
```

3

```
(define count-occurrences
  (lambda (s slst)
```

```
    (if (null? slst)
        0
        (+ (count-in-sexp s (car slst))
           (count-occurrences s (cdr slst))))))

(define count-in-sexp
  (lambda (s sexp)
    (if (symbol? sexp)
        (if (eq? s sexp) 1 0)
        (count-occurrences s sexp))))
```

4

```
(define flatten
  (lambda (slst)
    (if (null? slst)
        '()
        (append (flatten-sexp (car slst))
                (flatten (cdr slst))))))

(define flatten-sexp
  (lambda (sexp)
    (if (symbol? sexp)
        (list sexp)
        (flatten sexp))))
```

5

```
(define merge
  (lambda (lon1 lon2)
    (if (null? lon1)
        lon2
        (if (null? lon2)
            lon1
            (if (<= (car lon1) (car lon2))
```

```
                (cons (car lon1) (merge (cdr lon1) lon2))
                (cons (car lon2) (merge lon1 (cdr lon2))))))))
```

2.2.9

1

```
(define path
  (lambda (n bst)
    (if (null? bst)
        '()
        (if (= n (car bst))
            '()
            (if (< n (car bst))
                (cons 'l (path n (cadr bst)))
                (cons 'r (path n (caddr bst))))))))
```

4

```
(define compose
  (lambda procs
    (comp procs)))

(define comp
  (lambda (procs)
    (if (null? procs)
        (lambda (x) x)
        (if (null? (cdr procs))
            (car procs)
            (binary-compose (car procs) (comp (cdr procs)))))))

(define binary-compose
  (lambda (f g)
    (lambda (x)
      (f (g x)))))
```

5

```
(define sort
  (lambda (lon)
    (if (null? lon)
        '()
        (cons (lowest lon)
              (sort (remove-first-occurrence
                     (lowest lon)
                     lon))))))

(define lowest
  (lambda (lon)
    (lowest-iter (car lon) lon)))

(define lowest-iter
  (lambda (n lon)
    (if (null? lon)
        n
        (if (< (car lon) n)
            (lowest-iter (car lon) (cdr lon))
            (lowest-iter n (cdr lon))))))

(define remove-first-occurrence
  (lambda (n lon)
    (if (null? lon)
        '()
        (if (= n (car lon))
            (cdr lon)
            (cons (car lon)
                  (remove-first-occurrence n (cdr lon)))))))
```

6

```
(define sort
  (lambda (p lon)
```

```
    (if (null? lon)
        '()
        (cons (first p lon)
              (sort p
                    (remove-first-occurrence
                     (first p lon)
                     lon))))))

(define first
  (lambda (p lon)
    (first-iter p (car lon) lon)))

(define first-iter
  (lambda (p n lon)
    (if (null? lon)
        n
        (if (p (car lon) n)
            (first-iter p (car lon) (cdr lon))
            (first-iter p n (cdr lon))))))

(define remove-first-occurrence
  (lambda (n lon)
    (if (null? lon)
        '()
        (if (= (car lon) n)
            (cdr lon)
            (cons (car lon)
                  (remove-first-occurrence n (cdr lon)))))))
```

2.3.1

```
(define free-vars
  (lambda (exp)
    (no-duplicates (all-free exp '()))))
```

```
(define all-free
  (lambda (exp formal-parameters)
    (if (varref? exp)
        (if (member exp formal-parameters)
            '()
            (list exp))
        (if (lambda-exp? exp)
            (all-free (body exp)
                      (cons (formal-parameter exp)
                            formal-parameters))
            (append (all-free (operator exp) formal-parameters)
                    (all-free (operand exp) formal-parameters))))))

(define bound-vars
  (lambda (exp)
    (no-duplicates (all-bound exp '()))))
(define all-bound
  (lambda (exp formal-parameters)
    (if (varref? exp)
        '()
        (if (lambda-exp? exp)
            (if (member (formal-parameter exp)
                        (free-vars (body exp)))
                (cons (formal-parameter exp)
                      (all-bound (body exp)
                                 (cons (formal-parameter exp)
                                       formal-parameters)))
                (all-bound (body exp)
                           (cons (formal-parameter exp)
                                 formal-parameters)))
            (append (all-bound (operator exp) formal-parameters)
                    (all-bound (operand exp) formal-parameters))))))
```

```
(define varref? symbol?)
(define lambda-exp?
  (lambda (exp)
    (eq? (car exp) 'lambda)))
(define application? ; not used
  (lambda (exp)
    (= (length exp) 2)))
(define body caddr)

(define formal-parameter
  (lambda (exp)
    (car (cadr exp))))
(define operator car)
(define operand cadr)

(define no-duplicates
  (lambda (lst)
    (if (null? lst)
        '()
        (if (memq (car lst) (cdr lst))
            (no-duplicates (cdr lst))
            (cons (car lst) (no-duplicates (cdr lst)))))))
```

2.3.2 (extends 2.3.1) We stipulate that a variable having no references in an expression is not free. The same follows for a variable not being bound.

```
(define free?
  (lambda (var exp)
    (if (varref? exp)
        (eq? exp var)
        (if (lambda-exp? exp)
            (if (eq? var (formal-parameter exp))
                #f
```

```
                (free? var (body exp)))
            (or (free? var (operator exp))
                (free? var (operand exp)))))))

(define bound?
  (lambda (var exp)
    (if (varref? exp)
        #f
        (if (lambda-exp? exp)
            (if (eq? var (formal-parameter exp))
                (or (free? var (body exp))
                    (bound? var (body exp)))
                (bound? var (body exp)))
            (or (bound? var (operator exp))
                (bound? var (operand exp)))))))
```

2.3.3

```
'((lambda (x) x) x)
```

2.3.4

```
'((lambda (x) y) x)
```

2.3.5 (extends 2.3.1)

```
(define free-vars
  (lambda (exp)
    (no-duplicates (all-free exp '()))))
(define all-free
  (lambda (exp params)
    (if (varref? exp)
        (if (member exp params)
            '()
            (list exp))
        (if (lambda-exp? exp)
            (all-free (body exp)
```

```
                        (append (formal-parameters exp)
                                params))
                (append (all-free (operator exp) params)
                        (reduce append '()
                                (map (lambda (operand)
                                       (all-free operand params))
                                     (operands exp))))))))

(define bound-vars
  (lambda (exp)
    (no-duplicates (all-bound exp '()))))
(define all-bound
  (lambda (exp params)
    (if (varref? exp)
        '()
        (if (lambda-exp? exp)
            (append (occurrences (formal-parameters exp)
                                 (free-vars (body exp)))
                    (all-bound (body exp)
                               (append (formal-parameters exp)
                                       params)))
            (append (all-bound (operator exp) params)
                    (reduce append '()
                            (map (lambda (operand)
                                   (all-bound operand params))
                                 (operands exp))))))))

(define operands cdr) ; for applications
(define formal-parameters
  (lambda (exp)
    (if (null? (cadr exp)) ; no formal parameters
        '()
```

```
        (if (null? (cdadr exp)) ; only one formal parameter
            (list (caadr exp))
            (cadr exp))))) ; at least one formal parameter

(define occurrences
  (lambda (members lst)
    (if (null? members)
        '()
        (if (member (car members) lst)
            (cons (car members) (occurrences (cdr members) lst))
            (occurrences (cdr members) lst)))))
```

2.3.10 (extends 2.3.5)

```
(define lexical-address
  (lambda (exp)
    (transform-exp exp '() (free-vars exp))))
(define transform-exp
  (lambda (exp env free)
    (if (varref? exp)
        (list exp ': (depth exp env) (position exp env free))
        (if (if-exp? exp)
            (list 'if
                  (transform-exp (predicate exp) env free)
                  (transform-exp (consequent exp) env free)
                  (transform-exp (alternative exp) env free))
            (if (lambda-exp? exp)
                (list 'lambda
                      (formal-parameters exp)
                      (transform-exp
                       (body exp)
                       (extend (formal-parameters exp) env)
                       free))
                (cons (transform-exp (operator exp) env free)
```

```
                        (map (lambda (operand)
                               (transform-exp operand env free))
                             (operands exp))))))))

(define all-free
  (lambda (exp params)
    (if (varref? exp)
        (if (memq exp params)
            '()
            (list exp))
        (if (if-exp? exp)
            (append (all-free (predicate exp) params)
                    (all-free (consequent exp) params)
                    (all-free (alternative exp) params))
            (if (lambda-exp? exp)
                (all-free (body exp)
                          (append (formal-parameters exp)
                                  params))
                (append (all-free (operator exp) params)
                        (reduce append
                                '()
                                (map (lambda (l)
                                       (all-free l params))
                                     (operands exp)))))))))

(define if-exp?
  (lambda (exp)
    (eq? (car exp) 'if)))

(define predicate cadr)
(define consequent caddr)
(define alternative cadddr)
```

```
(define extend
  (lambda (params env)
    (if (null? env)
        (list params)
        (cons params env))))

(define depth
  (lambda (var env)
    (depth-iter 0 var env)))
(define depth-iter
  (lambda (d var env)
    (if (null? env)
        d
        (if (member var (car env))
            d
            (depth-iter (+ 1 d) var (cdr env))))))

(define position
  (lambda (var env free)
    (if (null? env)
        (place var free)
        ((lambda (p)
           (if (eq? p 'not-here)
               (position var (cdr env) free)
               p))
         (place var (car env))))))
(define place
  (lambda (var los)
    (place-iter 0 var los)))
(define place-iter
  (lambda (n var los)
```

```
    (if (null? los)
        'not-here
        (if (eq? var (car los))
            n
            (place-iter (+ n 1) var (cdr los))))))
```

2.3.12

```
(lambda (x)
  (lambda (y)
    x))
```

2.3.13 (extends 2.3.10)

The solution applies only to expressions having no free variables. We could generate new variables, but we shall cope by saying no such expression exists, as the exercise says, for non-combinators.

To substitute, we fetch the variable in the environment corresponding to the given lexaddr, if it exists. If so, then we must determine this variable shadows all other variables of the same name. To do this, we convert the variable into a lexaddr, which denotes the most shallow occurrence of the variable in the environment. `correct-scope?` checks if the given lexaddr and the correct lexaddr (by `get-lexaddr`) are equal.

```
(define un-lexical-address
  (lambda (exp)
    (transform-exp exp '())))
(define transform-exp
  (lambda (exp env)
    (if (varref? exp)
        (substitute exp env)
        (if (if-exp? exp)
            (let ((p-result (transform-exp (predicate exp) env))
                  (c-result (transform-exp (consequent exp) env))
                  (a-result (transform-exp (alternative exp) env))
              (if (not (or p-result c-result a-result))
```

```
                    #f
                    (cons 'if (append p-result c-result a-result))))
              (if (lambda-exp? exp)
                  (let ((result
                         (transform-exp
                          (body exp)
                          (extend (formal-parameters exp) env))))
                    (if result
                        (list 'lambda
                              (formal-parameters exp)
                              result)
                        #f))
                  (let ((result (map (lambda (l)
                                       (transform-exp l env))
                                     (extend (operator exp)
                                             (operands exp)))))
                    (if (member #f result)
                        #f
                        result)))))))

(define substitute
  (lambda (lexaddr env)
    (if (null? env)
        #f ; free var
        (let ((frame
               (frame-at-index
                env
                (lexaddr-depth lexaddr))))
          (if frame
              (let ((var
                     (var-at-index
                      frame
```

```
                      (lexaddr-position lexaddr))))
                (if var
                    (if (correct-scope?
                         (get-lexaddr var env) lexaddr)
                        var
                        #f)
                    #f))
              #f)))))

(define varref?
  (lambda (exp)
    (eq? (car exp) ':)))

(define correct-scope? lexaddr=?)

(define lexaddr-depth cadr)
(define lexaddr-position caddr)
(define lexaddr=?
  (lambda (a b)
    (if (= (lexaddr-depth a) (lexaddr-depth b))
        (= (lexaddr-position a) (lexaddr-position b))
        #f)))

(define position
  (lambda (sym list-of-los)
    (if (null? list-of-los)
        #f
        (let ((n (place sym (car list-of-los))))
          (if (eq? n 'not-here)
              (position sym (cdr list-of-los))
              n)))))
```

```
(define var-at-index safe-list-ref)
(define frame-at-index safe-list-ref)
(define safe-list-ref
  (lambda (l n)
    (if (null? l)
        #f
        (if (= 0 n)
            (car l)
            (safe-list-ref (cdr l) (- n 1))))))
```

2.3.14 (extends 2.3.2)

$exp[y/x] \equiv exp[x \leftarrow\!\shortmid\, y]$, `(rename exp x y)`

```
(define rename
  (lambda (exp var1 var2)
    (if (free? var1 exp)
        #f
        (rename-i exp var1 var2))))

(define rename-i
  (lambda (exp var1 var2)
    (if (varref? exp)
        (if (eq? exp var2) var1 exp)
        (if (lambda-exp? exp)
            (if (eq? (formal-parameter exp) var1)
                exp
                (if (eq? (formal-parameter exp) var2)
                    exp
                    (list 'lambda
                          (list (formal-parameter exp))
                          (rename-i (body exp) var1 var2))))
            (list (rename-i (operator exp) var1 var2)
                  (rename-i (operand exp) var1 var2))))))
```

2.3.15 (extends 2.3.14)

```
(define alpha-convert
  (lambda (exp v)
    (let ((renamed-exp
           (rename (body exp) v (formal-parameter exp))))
      (if renamed-exp
          (list 'lambda (list v) renamed-exp)
          #f))))
```